

ELTE EÖTVÖS LORÁND UNIVERSITY

DEPLOYMENT, SCALING, CACHING

Modern web application development in .NET

Lecture 13

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

DEPLOYING ASP.NET APPLICATIONS

After development, web pages must be *deployed* to a web server

- The configuration (**`appsettings.json`**) can be specified for a *development*, *staging* (testing) and *production* version (**`appsetting.<environment>.json`**)
- The environment configuration can be controlled by the value of the **`ASPNETCORE_ENVIRONMENT`** environment variable
  - when not specified, the default value is *production*;
  - the value can also be specified for each runtime profile in the **`launchSettings.json`** file, with the default value being *development*.

DEPLOYING ASP.NET APPLICATIONS

- Usual configuration setups:
  - **Development:** Enables detailed error pages, Webpack hot reloading, runtime Razor compilation, and more.
  - **Staging:** Used for final testing before production. Similar to *production* but may include extra logging or test data.
  - **Production:** Optimized for performance and security. Disables development features such as detailed exceptions.
- *Razor views* (**`.cshtml`** files) are also compiled by default when the project is compiled or deployed. Runtime recompilation can also be enabled, and is automatically enabled when using Visual Studio.

```
builder.Services.AddControllersWithViews()
                .AddRazorRuntimeCompilation();
```

DEPLOYING ASP.NET APPLICATIONS

The .NET Core framework is platform-independent, so the compilation and execution steps are also available as terminal commands:

- Use the **`dotnet run`** command to compile and run your .NET Core project.
- Use the **`dotnet build`** command to compile only.
- The **`dotnet publish -o out_dir`** command places the .NET Core project in the specified directory, copying all compiled binary files, configuration files, and dependencies to a single location. The web application can then be run without the SDK, only the .NET Core Runtime is required: **`dotnet MyWebApp.dll`**

Webszolgáltatások kihelyezése

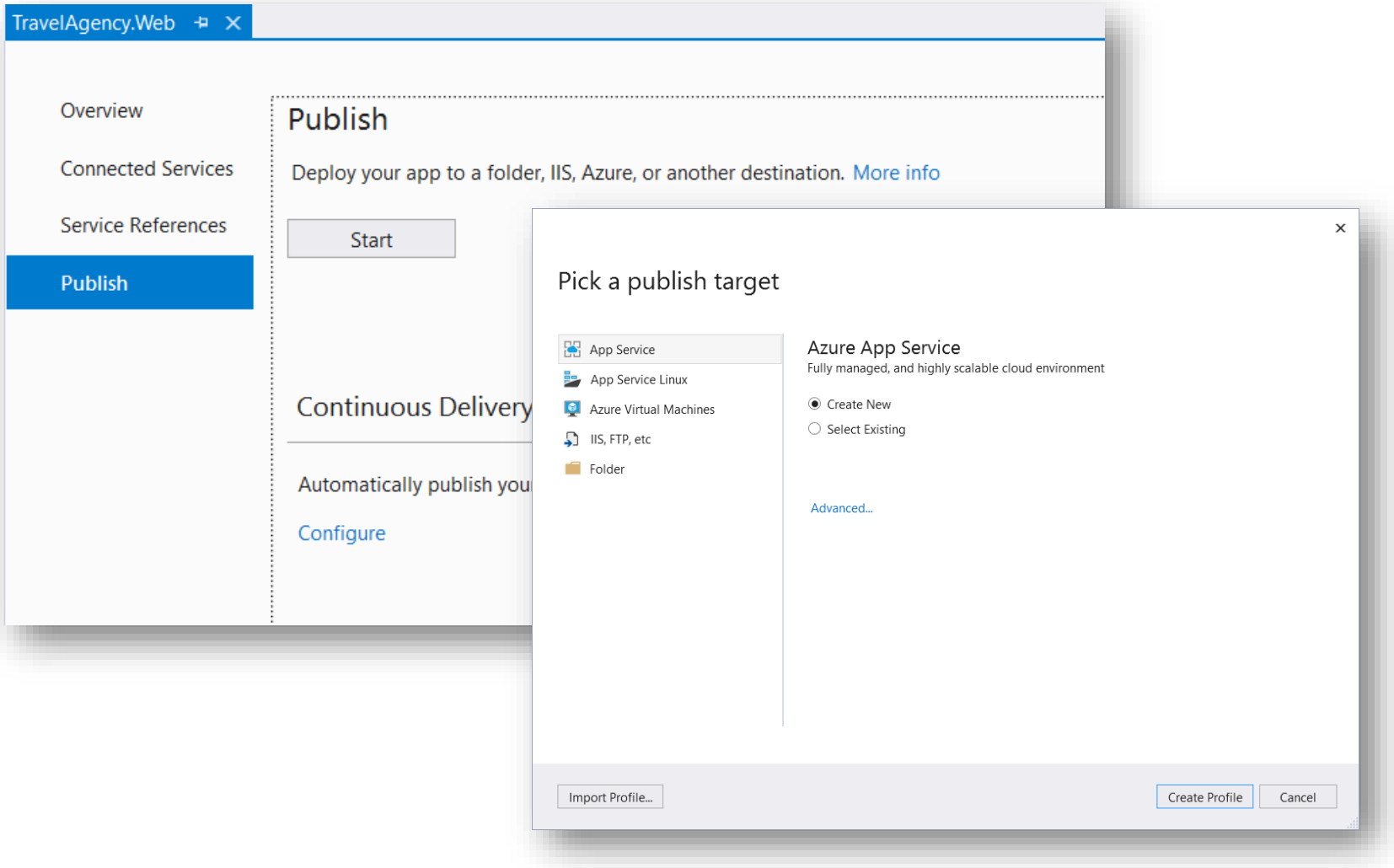
Kihelyezés a Microsoft Azure felhőjébe

- Díjszabás használat szerint
  - Ingyenes 170 € értékű kredit regisztrációkor.

	INGYENES Ingyenes próbalehetőség	KÖZÖS Fejlesztő- és tesztkörnyezet	ALAPSZINTŰ Dedikált fejlesztő- és tesztkörnyezet	STANDARD Futtatás éles környezetben	PRÉMIUM Jobb teljesítmény és méretezés	IZOLÁLT Nagyobb teljesítmény, biztonság és elszigeteltség
Webes, mobil- vagy API-alkalmazások	10	100	Korlátlan	Korlátlan	Korlátlan	Korlátlan
Lemezterület	1 GB	1 GB	10 GB	50 GB	250 GB	1 TB
Maximális példányszám	–	–	Akár 3	Akár 10	Akár 30**	Akár 100*
Egyéni tartomány	–	Támogatott	Támogatott	Támogatott	Támogatott	Támogatott
Automatikus méretezés	–	–	–	Támogatott	Támogatott	Támogatott
Hibrid VPN-kapcsolat	–	–	–	Támogatott	Támogatott	Támogatott
Hálózati elszigeteltség						Támogatott
Ár	Ingyenes	€0,011/ óra	€0,064/ óra	€0,085/ óra	€0,169/ óra	€0,253/ óra

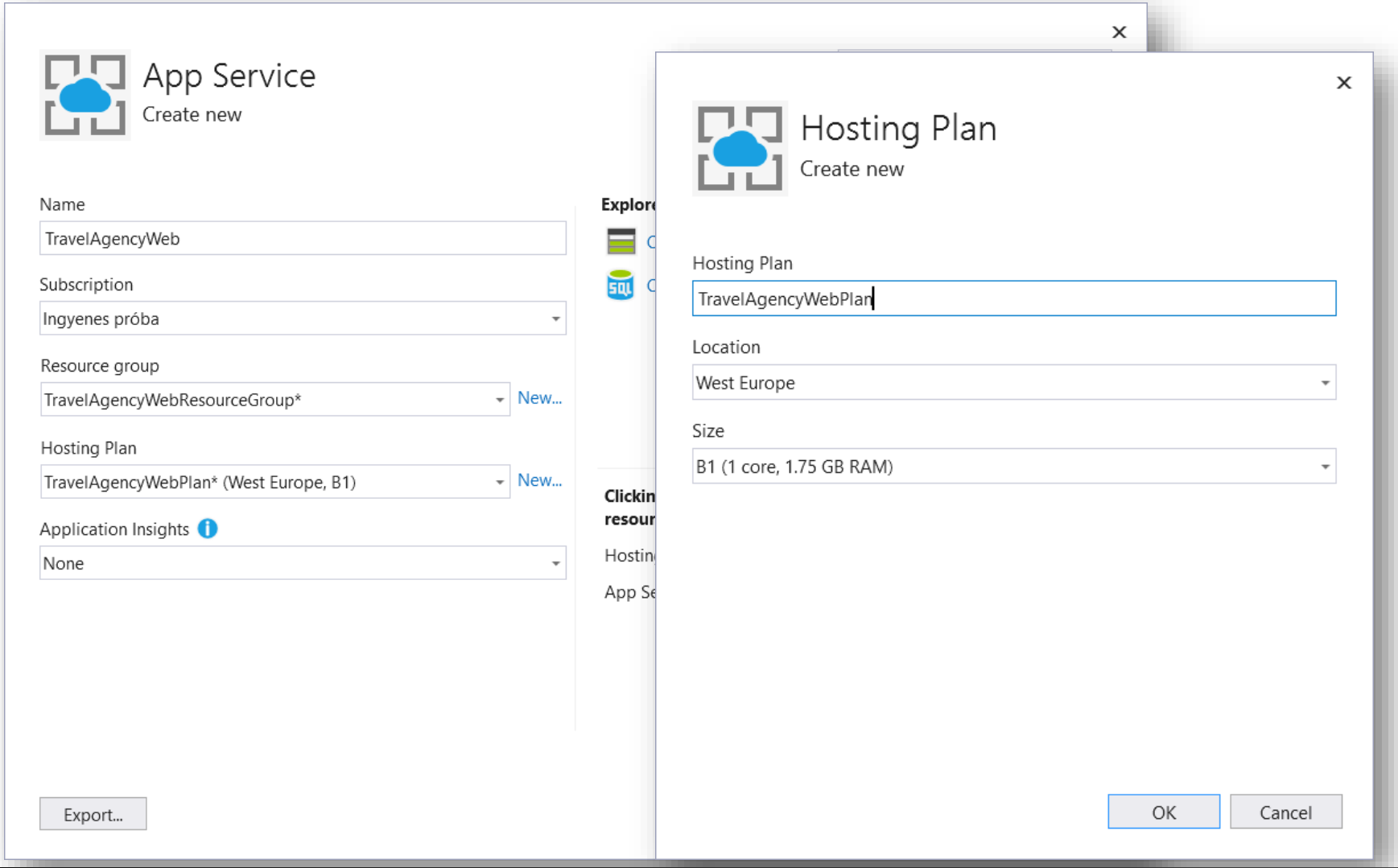
Webszolgáltatások kihelyezése

Kihelyezés a Microsoft Azure felhőjébe



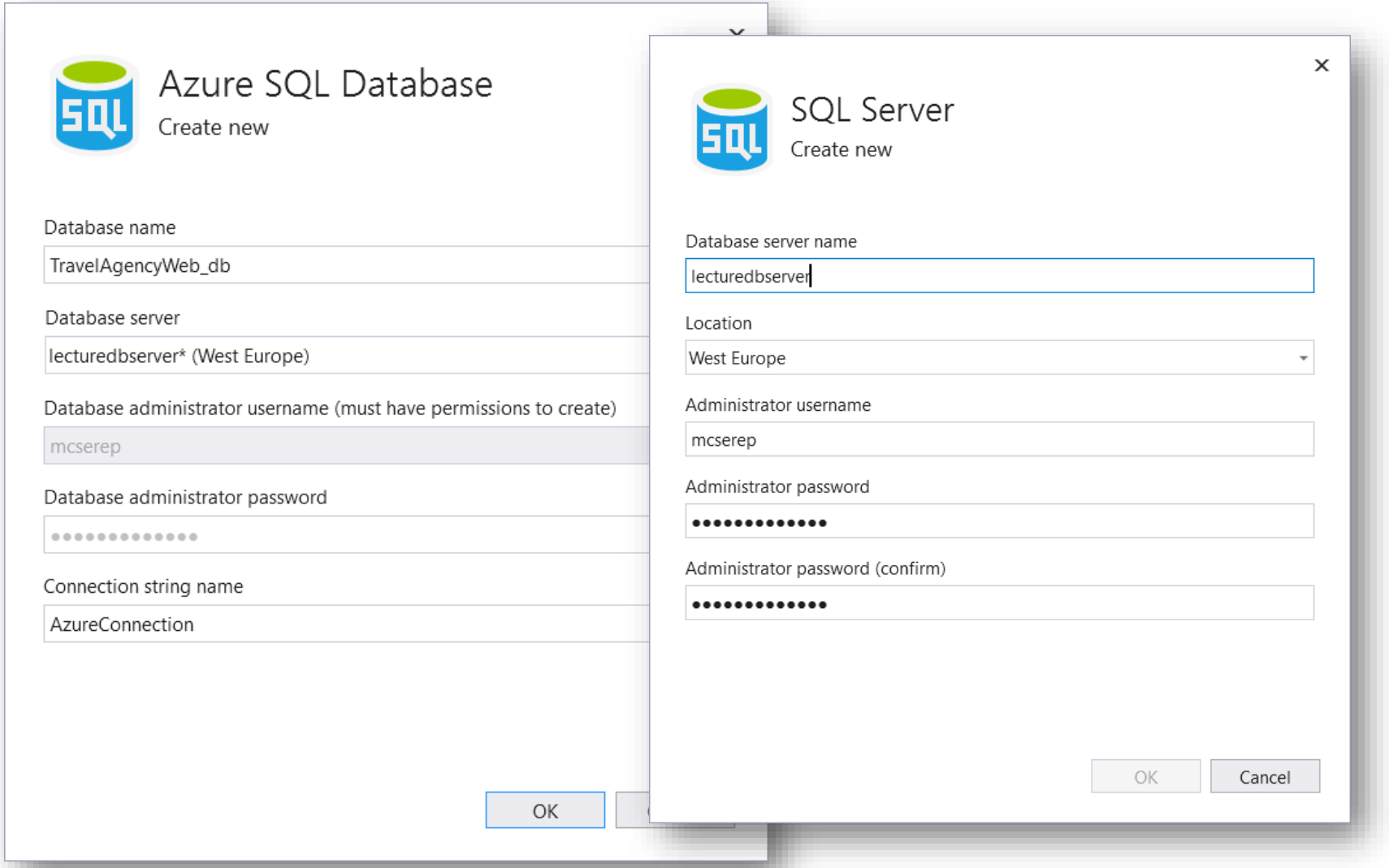
Webszolgáltatások kihelyezése

Kihelyezés a Microsoft Azure felhőjébe



Webszolgáltatások kihelyezése

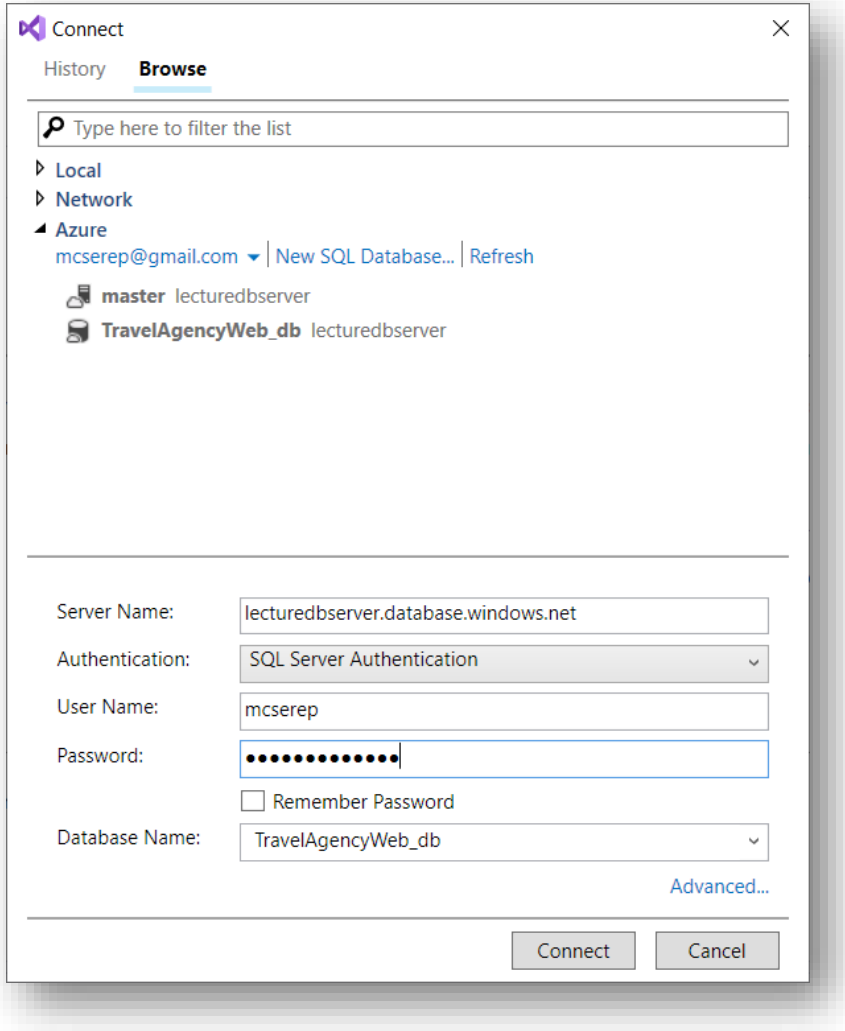
Kihelyezés a Microsoft Azure felhőjébe



Webszolgáltatások kihelyezése

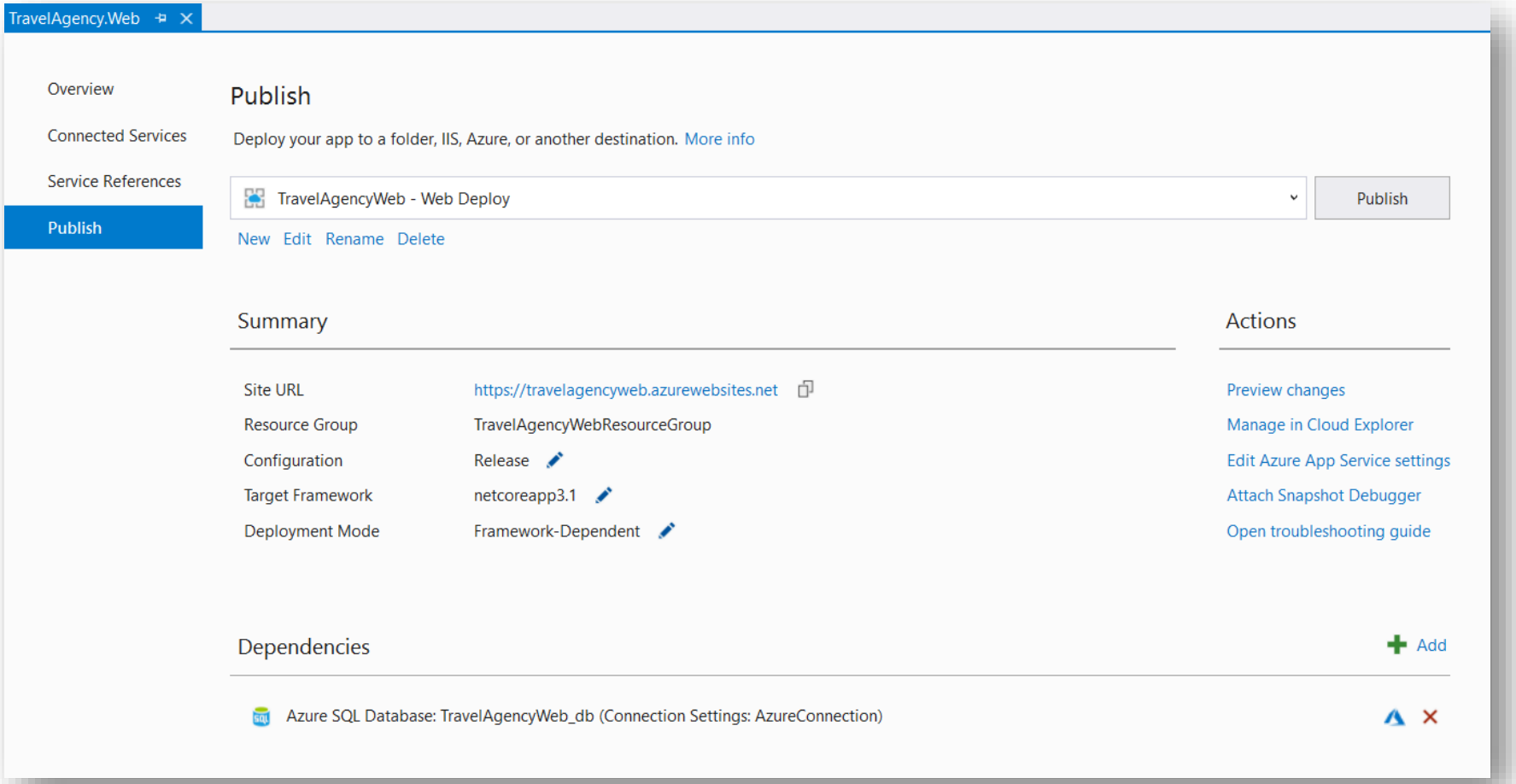
Kihelyezés a Microsoft Azure felhőjébe

- *SQL Server Object Explorer -> Add SQL Server -> Azure*
  - Amennyiben Azure fiókunkkal vagyunk bejelentkezve, egyből tallózhatunk a távoli adatbázisok között.
- Hasonló módon konfigurálható az SQL Server Management Studio-ban is a kapcsolat.



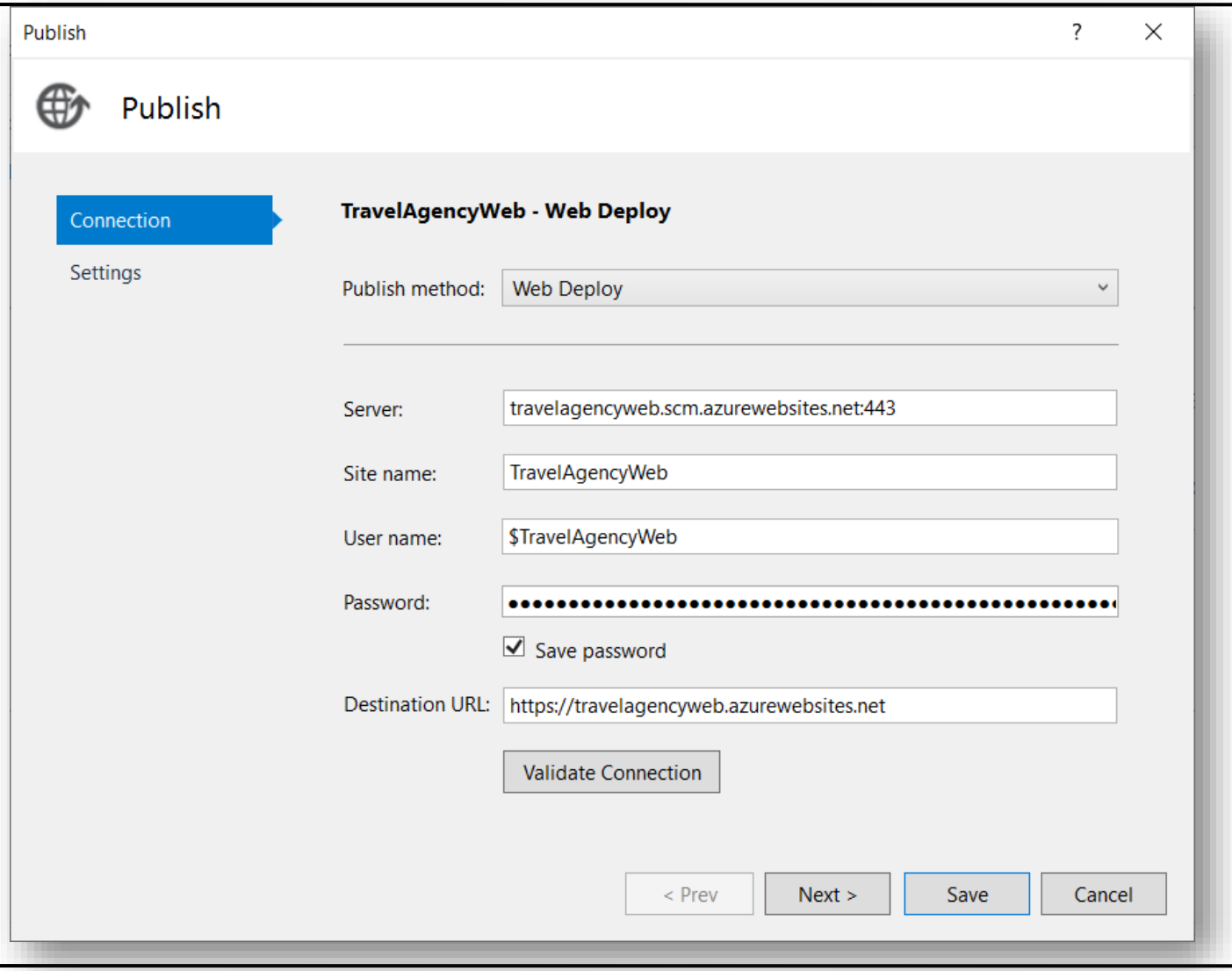
Webszolgáltatások kihelyezése

Kihelyezés a Microsoft Azure felhőjébe



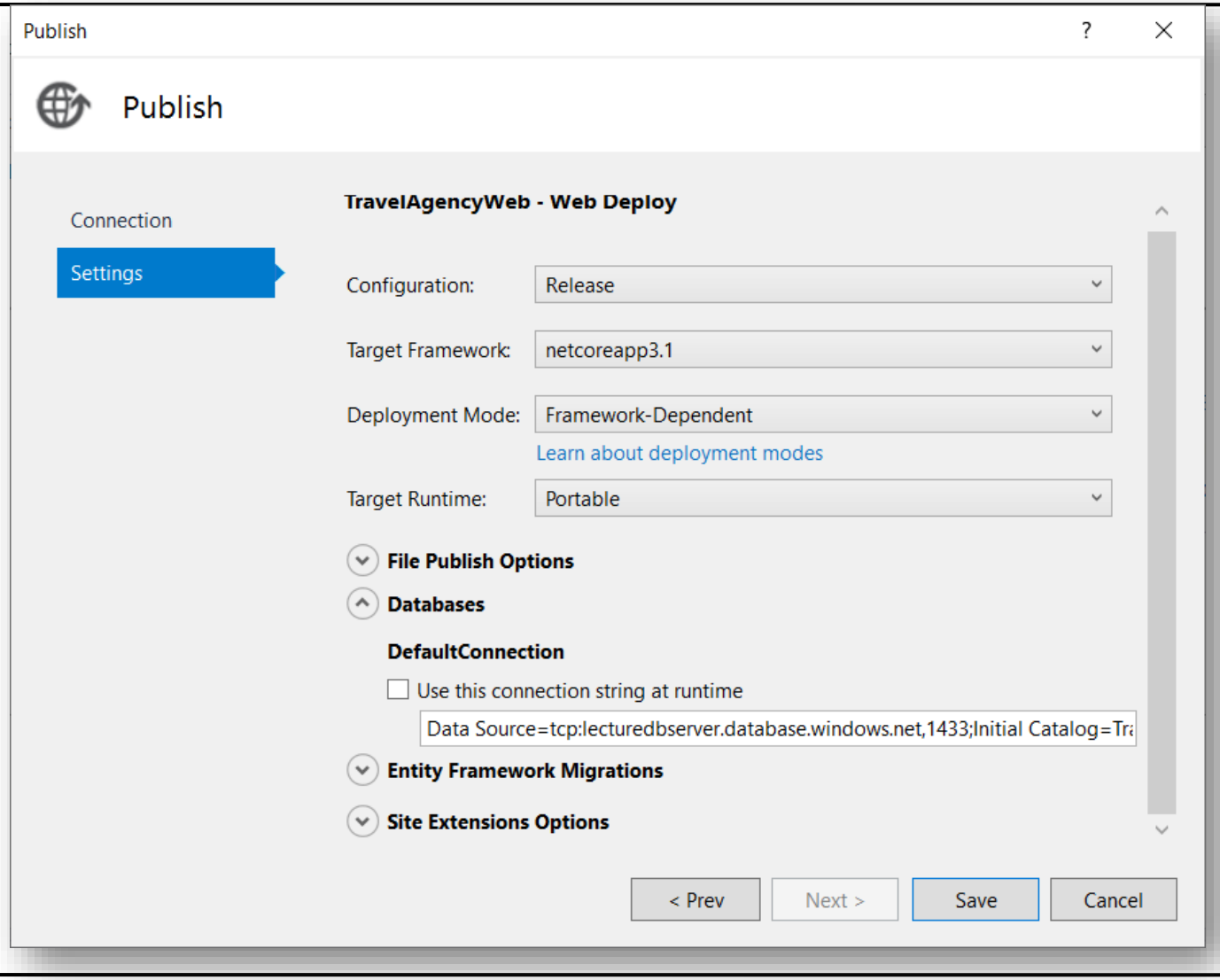
Webszolgáltatások kihelyezése

Kihelyezés a Microsoft Azure felhőjébe



Webszolgáltatások kihelyezése

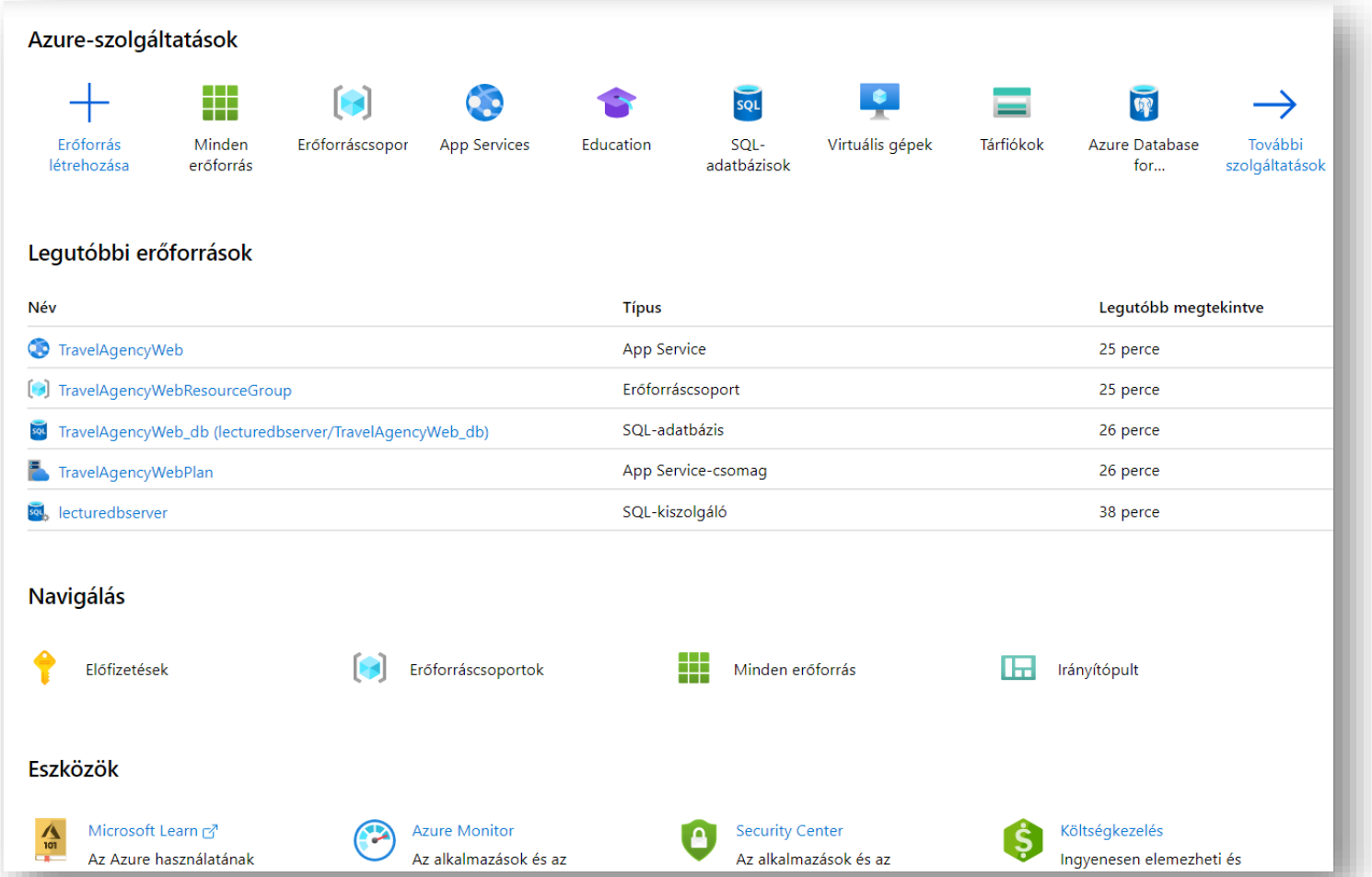
Kihelyezés a Microsoft Azure felhőjébe



Webszolgáltatások kihelyezése

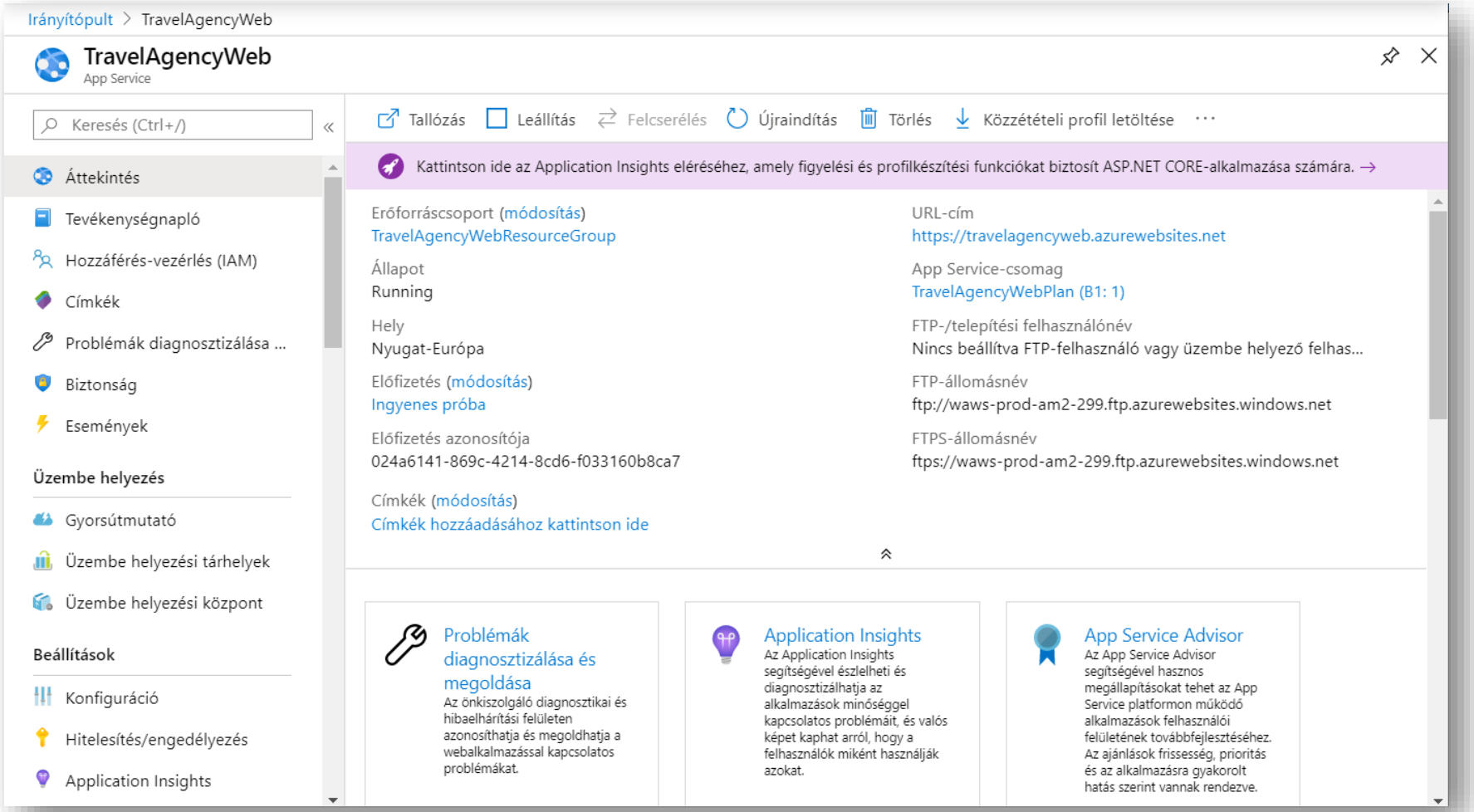
Microsoft Azure Portal

- Microsoft Azure Portal: https://portal.azure.com



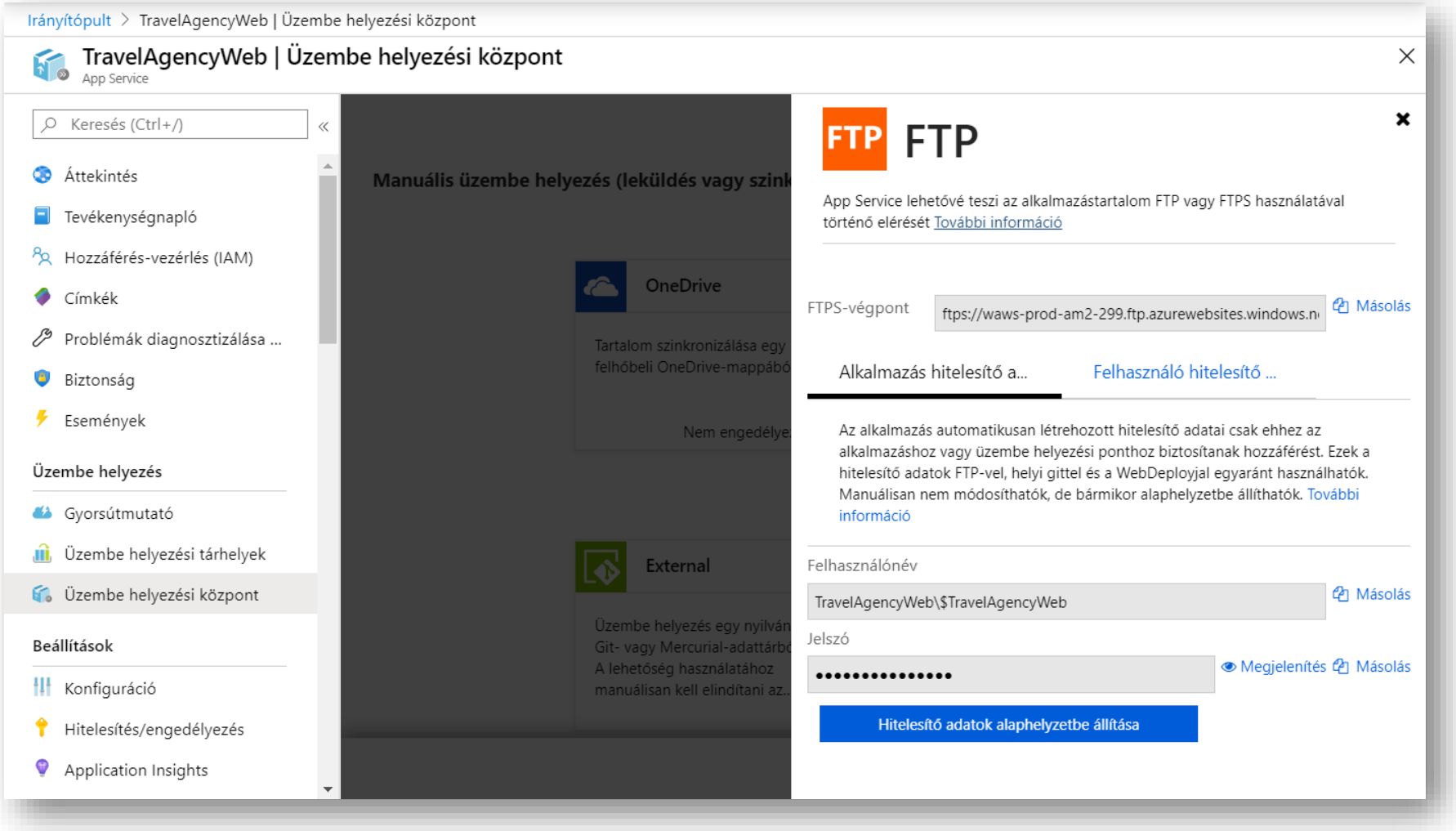
Webszolgáltatások kihelyezése

Microsoft Azure Portal



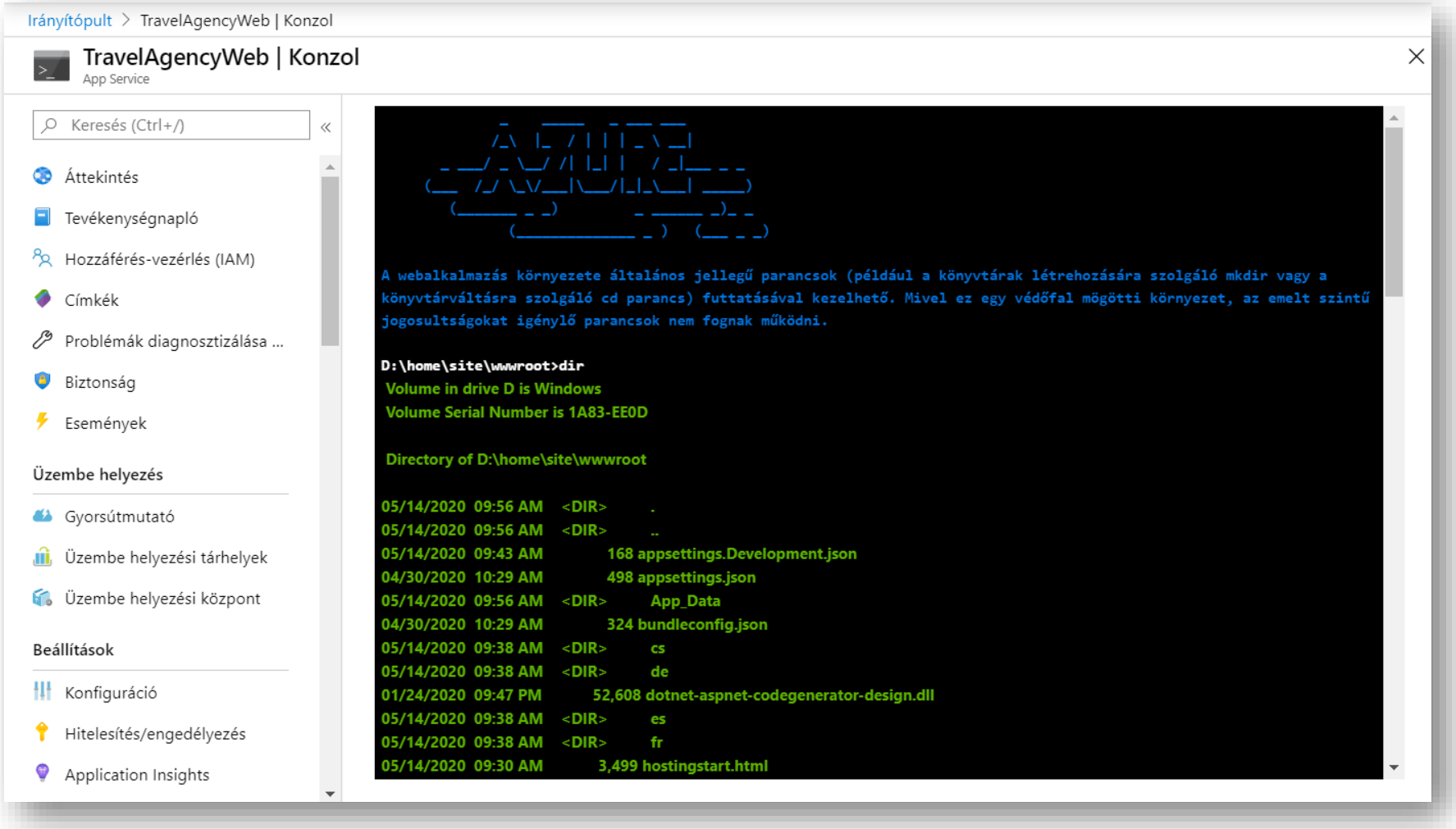
Webszolgáltatások kihelyezése

Microsoft Azure Portal



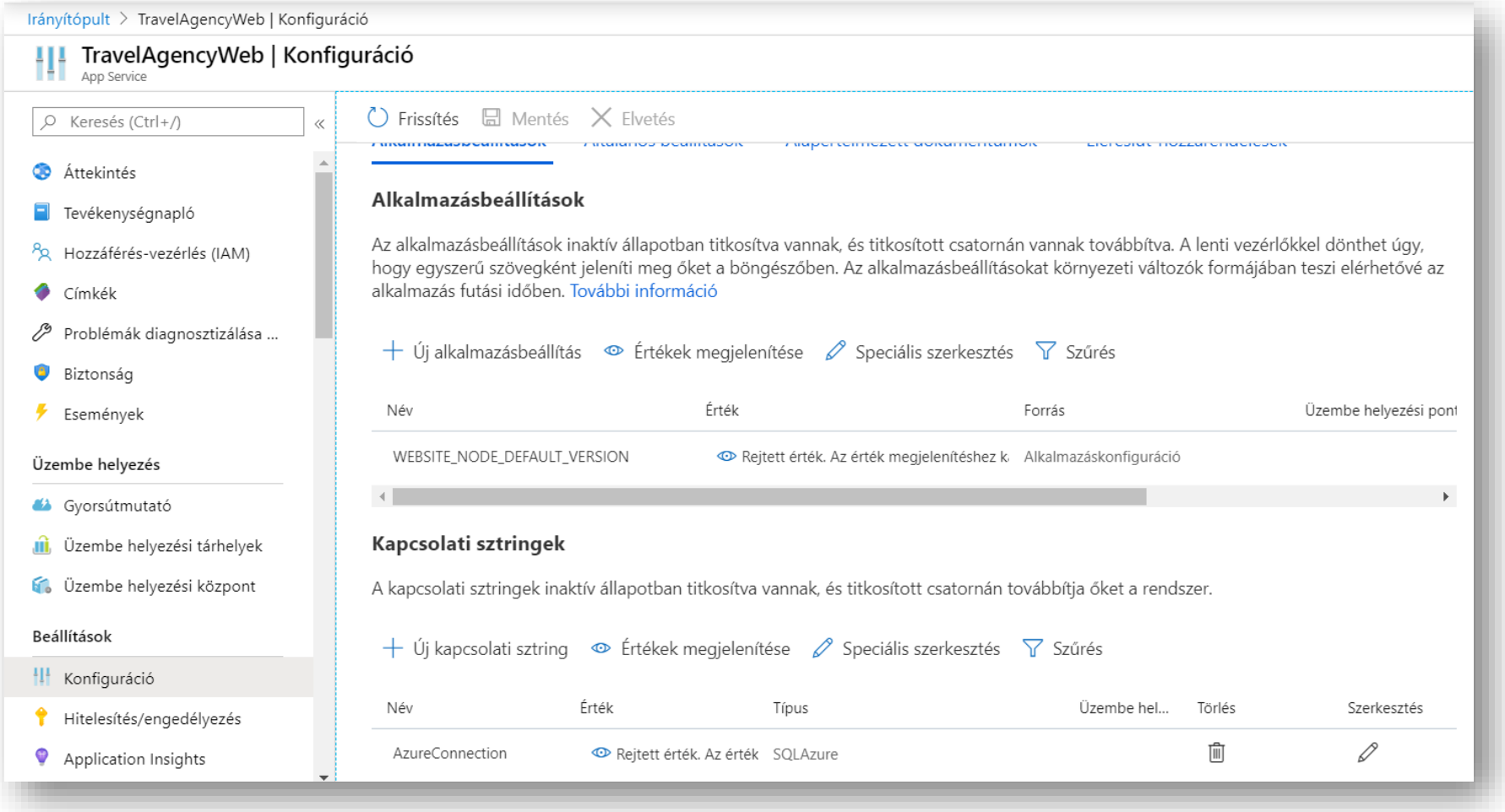
Webszolgáltatások kihelyezése

Microsoft Azure Portal



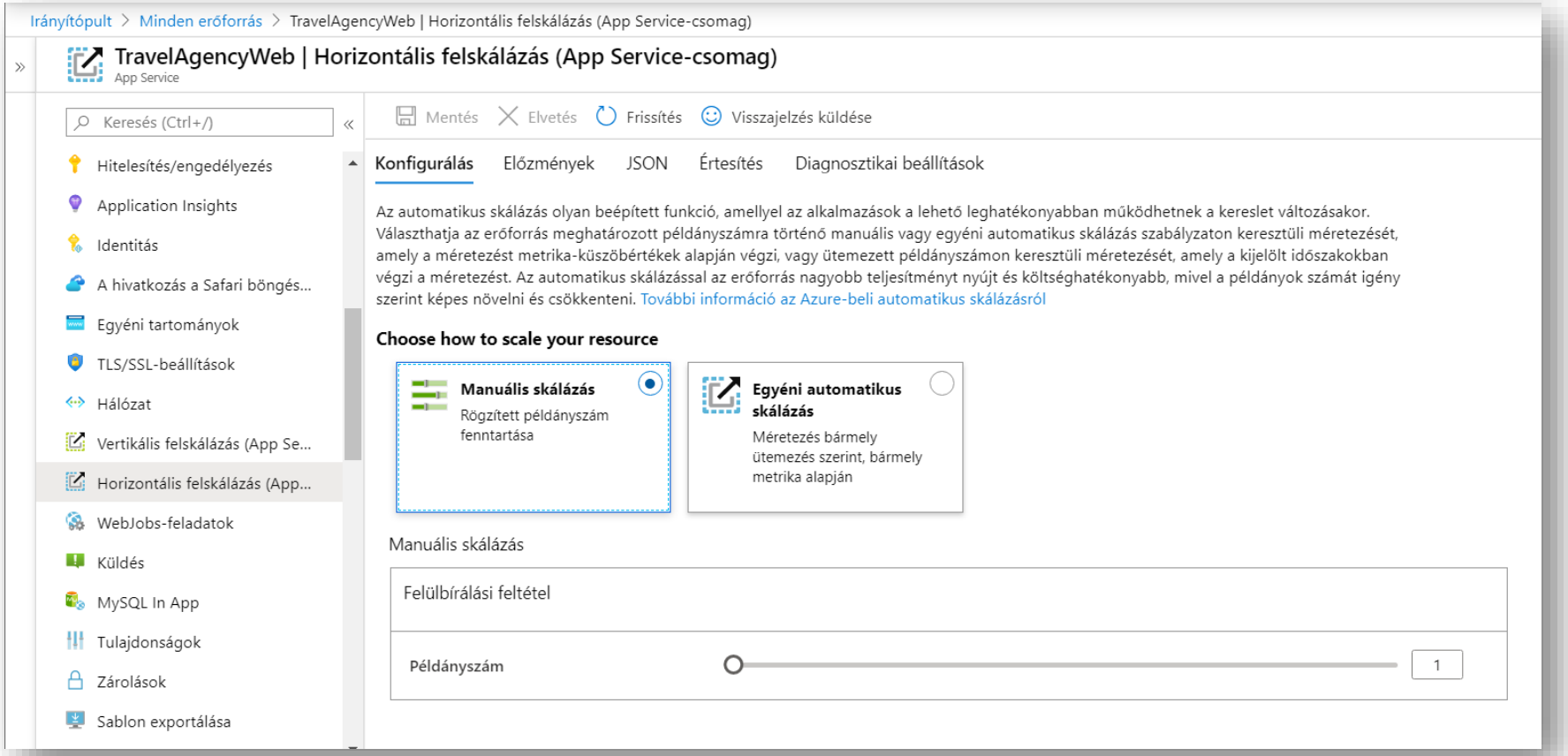
Webszolgáltatások kihelyezése

Microsoft Azure Portal



Webszolgáltatások kihelyezése

Microsoft Azure Portal



SCALING SIGNALR

Why scale a SignalR application?

- SignalR uses long-lived connections
  - Each client maintains a persistent WebSocket (or similarly an SSE or a long polling) connection
  - These connections remain open for the entire user session
- Single server approach has limitations
  - One server can handle typcially only a few thousand concurrent connections
  - Resources (memory, CPU, threads, sockets) are quickly exhausted
  - No redundancy — if it fails, all clients disconnect

Horizontal scaling (multiple servers) is necessary!

SCALING SIGNALR

Horizontally scaling an application

- Deploy multiple application server instances
- Use a load balancer to distribute incoming client connections
- Enables high availability and load distribution

Horizontally scaling a SignalR app

- SignalR hubs are in-memory by default
- Scaling out requires all nodes to share connection information
- Sticky sessions (client always goes to same server) are not reliable or scalable
  - Goes against the very purpose of load balancing
  - Doesn’t handle failover

SCALING SIGNALR

To share state across servers, a custom backplane (e.g. Redis) or *Azure SignalR Service* should be used.

Tasks of the backplane:

- Broadcast messages across all instances
- Share group membership and connection data

Benefits of Redis backplane:

- High performance, low latency
- In-memory and distributed
- Supports publish / subscribe for message broadcasting
- Simplifies state sharing and group membership tracking across nodes

CACHING WITH REDIS

Using Redis as a backplane

- Redis can synchronize messages between servers
- Acts as a message bus for SignalR across a web farm
- Install Redis, then can be easily used with the **`StackExchange.Redis`** NuGet package, which implements a Redis client
- Configuration:

```
builder.Services.AddStackExchangeRedisCache(options => {
  options.Configuration = "localhost:6379",
  options.InstanceName = "MyApp";
});
```

- Inject an **`IDistributedCache`** interface where caching is required

AZURE SIGNALR SERVICE

Azure SignalR for provides auto-scaling for SignalR applications

- Fully managed real-time messaging service.
- Handles connection management, scaling, and message delivery.
- Ideal for applications with unpredictable or high traffic.
- Requires hosting the application in the Azure cloud:
  - Create an Azure SignalR resource on the Azure portal or with the Azure CLI (see the **`az signalr create`** command).
  - Save the generated URI to *Azure:SignalR:ConnectionString* environment variable.
  - Enable Azure SignalR Services in your app: **`builder.Services.AddSignalR().AddAzureSignalR();`**